

UNICC AI SAFETY COUNCIL

SLM Platform — Individual Report

Jun Kim · SLM Platform Lead

Submitted to:

New York University
Master of Science in Management and Systems
MASY GC-4100 — Applied Technical Project Capstone

Course Instructor:

Professor Joseph Ng

Project Sponsor:

Dr. Andres G. Fortino | Anusha Dandapan, UNICC

Project Team:

Jun Kim (SLM Platform Lead) · Wesley Koe (Council of Experts) · Dennis Wu (Frontend UI)

Submission Date:

April 21, 2026

GitHub: <https://github.com/wesleykoe/UNICC-Capstone>

Live Demo: <https://unicc-ai-guard.replit.app>

Executive Summary

This individual report documents my contributions as SLM Platform Lead on the UNICC AI Safety Council capstone project. My primary responsibility was the design, implementation, and deployment of the backend evaluation engine — the system that transforms an AI application submission into a structured, evidence-backed safety verdict.

The platform implements a Council-of-Experts architecture in which three independent AI expert agents (Governance, Threat, Behavioral) evaluate an AI application sequentially, engage in a structured deliberation process, and produce a final verdict via a deterministic arbitration rules module. The entire pipeline is orchestrated through a FastAPI backend I designed and built, running Llama 3.2-3B-Instruct locally with Claude API (claude-sonnet-4-5) as a configurable alternative backend.

This report covers my individual technical contributions in detail: the orchestration architecture, expert agent prompt engineering, deliberation layer design, Arbitration Rules v1.1, backend configuration system, deployment infrastructure, and key technical challenges resolved during development. Team integration context — Wesley Koe's Council of Experts framework and Dennis Wu's frontend — is summarized briefly to provide project context.

My Role	SLM Platform Lead
Primary Deliverable	FastAPI backend — full evaluation pipeline
Expert Agents Built	Governance, Threat, Behavioral (prompt engineering + schema)
Arbitration System	Rules v1.1 — 6 deterministic rules in priority order
Model Backend	Llama 3.2-3B-Instruct (local) + Claude API (configurable)
Deployment	Docker + docker-compose + DGX-ready requirements
Commits	159 total commits — GitHub: wesleykoe/UNICC-Capstone
Demo	April 14, 2026 — NYU UNICC Capstone Demo Day

1. Project Context and My Role

1.1 Project Overview

The UNICC AI Safety Council is a multi-agent SLM evaluation platform built for the United Nations International Computing Centre (UNICC), which provides shared ICT services to over 60 UN and international organizations worldwide. The platform addresses the absence of a systematic, automated mechanism to evaluate AI applications for safety and governance compliance before deployment within the UN common system.

The team comprised three members with distinct responsibilities: I led the SLM Platform (backend engine, orchestration, model integration, deployment); Wesley Koe designed the Council of Experts framework (expert schemas, arbitration rule specification, training data); and Dennis Wu built the frontend interface (Replit app, input/output UI). While the platform functions as an integrated system, my report focuses on the backend infrastructure I was solely responsible for building.

1.2 My Scope of Work

As SLM Platform Lead, my responsibilities covered everything from the HTTP layer down to model inference:

- FastAPI application design and implementation (app/slm/api.py)
- Expert agent orchestration — sequential pipeline management (Governance → Threat → Behavioral)
- Expert agent prompt engineering — translating Wesley's framework specifications into working LLM prompts
- Deliberation layer — critique and defense round logic and output schema
- Arbitration Rules v1.1 — implementing Wesley's rule specification as a deterministic decision engine
- Configurable backend system — Llama 3.2-3B-Instruct local inference + Claude API integration
- Schema validation — Pydantic v2 input/output validation with fallback handling

- Docker containerization and DGX deployment architecture
- ngrok tunnel configuration for Dennis's frontend integration
- Clean-machine deployment reliability (dotenv handling, README run commands)

2. Technical Architecture — What I Built

2.1 System Overview

The evaluation pipeline I built processes a request through four stages: input ingestion and validation, sequential expert agent analysis, deliberation, and arbitration. Every stage is orchestrated by `api.py`, which I designed as a single-file FastAPI application to keep the codebase portable and auditable.

Stage	What I Built
Input Layer	Schema v2.0 Pydantic validation. <code>/evaluate</code> and <code>/run</code> endpoints. Auto-detection of AI framework, model API calls, and auth configuration from GitHub URL.
Expert Pipeline	Sequential orchestration: Governance → Threat → Behavioral. Each agent uses a domain-specific system prompt I engineered. Outputs validated against frozen <code>expert_output_schema v1.0</code> .
Deliberation	Critique round + defense round logic. Each agent reviews others' findings, challenges reasoning, and defends its own position. Structured output schema with <code>position_changed</code> and <code>agree</code> flags.
Arbitration	Rules v1.1 — 6 deterministic rules applied in priority order to post-deliberation expert positions. Produces final verdict: Approve / Revise / Escalate / Reject.

2.2 FastAPI Backend — `api.py`

The core of my work lives in `app/slm/api.py`. This file contains the full orchestration logic, all expert agent prompts, the deliberation engine, and the arbitration rules module. Key design decisions I made:

Modular Backend Routing

I designed `generate_text_backend()` as a single dispatch function that routes inference to either the local Llama pipeline or the Claude API based on the `LLM_BACKEND` environment variable. This means the entire evaluation pipeline — prompts, deliberation, arbitration — is backend-agnostic. Swapping models requires no code changes, only a config change in `.env`.

LLM_BACKEND	Behavior
local	Runs Llama 3.2-3B-Instruct via HuggingFace Transformers pipeline. MPS acceleration on Apple M1, CUDA on DGX. ~30–90s per full evaluation.
anthropic	Routes to Claude API (claude-sonnet-4-5) via authenticated httpx POST. ~10–20s per evaluation. Used during development for prompt validation.

Sequential Expert Orchestration

The three expert agents run sequentially: Governance first, then Threat, then Behavioral. Each agent receives the same input payload but a distinct system prompt encoding its evaluation framework. I engineered each prompt to produce structured JSON output conforming to `expert_output_schema v1.0` — a schema Wesley and I co-defined. All inference calls use `temperature=0` and `seed=42` for deterministic, reproducible outputs.

Fallback and Resilience

If any expert agent returns malformed JSON or fails schema validation, the `failure_detected=true` flag is set and safe default values are applied for that agent's output fields. The pipeline never crashes — it always returns a complete `RunResponse`, even under partial failure conditions. This was a deliberate design decision to ensure evaluation continuity in production.

Graceful Dotenv Handling

During testing I identified a clean-machine deployment failure: importing `python-dotenv` would raise an `ImportError` if the package was not installed, crashing the server before any request was processed. I resolved this by wrapping the import in a `try/except` block, so the server starts successfully regardless of whether `python-dotenv` is installed. If it is installed, `.env` values are loaded; if not, the server falls back to system environment variables.

```
try:    from dotenv import load_dotenv    load_dotenv() except
ImportError:    pass
```

2.3 Expert Agent Prompt Engineering

Each expert agent's behavior is entirely determined by its system prompt — a structured instruction set I engineered to translate policy framework requirements into consistent, schema-compliant LLM outputs. The challenge was that a 3B parameter model has limited context capacity, so prompts needed to be precise enough to produce structured JSON reliably while encoding sufficient domain knowledge to produce meaningful evaluations.

Agent	Evaluation Focus	Frameworks Encoded in Prompt
Governance	Policy compliance, data governance, institutional mandate, accountability	UN AI Ethics Guidelines, EU AI Act Article 9, UNESCO AI Recommendation 2021
Threat	Attack vectors, prompt injection, API exposure, file upload exploits	NIST CSF 2.0 RS.MI-1, ISO 27001 A.8.2, MITRE ATT&CK T1190
Behavioral	Output consistency, alignment drift, intent fidelity, adversarial predictability	IEEE Ethically Aligned Design, OECD AI Principles 1.4, ACM Code of Ethics 1.7

I validated prompt quality by running both backends against identical inputs and comparing output structure and reasoning quality. Claude API (claude-sonnet-4-5) served as the benchmark during development — once prompt quality was confirmed against Claude, the same prompts were applied to Llama 3.2-3B-Instruct and outputs were confirmed structurally consistent.

2.4 Deliberation Layer

The Deliberation Layer is the feature I am most proud of in this project. Rather than averaging or aggregating independent agent scores — which would suppress genuine disagreements — I implemented a two-round peer review process modeled on academic peer review:

- Critique Round: Each expert agent receives the full outputs of the other two agents and produces a structured critique — identifying contradictions, gaps in reasoning, and

assessments that conflict with its own framework. The critique schema captures: challenge_summary, challenge_type, and agree flag (0/1).

- **Defense Round:** Each expert agent receives the critiques directed at its own output and produces a defense — either reaffirming its position with additional rationale or revising its assessment. The defense schema captures: response_summary, updated_recommended_action, position_changed flag (0/1), and confidence level.

This design means that if the Threat agent flags an unauthenticated API endpoint as a critical risk but the Governance agent initially classified it as a low-priority compliance issue, the deliberation process forces both agents to directly address that contradiction before arbitration. The final verdict reflects the deliberated consensus, not the initial independent scores.

2.5 Arbitration Rules v1.1

The Arbitration module is a deterministic decision engine I implemented based on the rule specification Wesley designed. It takes the post-deliberation expert positions as input and applies six rules in strict priority order:

Rule	Condition	Decision
1	Any expert severity_ranking = Critical	Escalate + Human Review Required
2	Threat overall_status = Fail	Reject
3	Governance overall_status = Fail	Revise + Human Review Required
4	Threat exploitability = High AND Behavioral overall_status = Fail	Reject
5	All experts overall_status = Pass	Approve
6	All other combinations	Revise

The triggered rule is always surfaced in the RunResponse alongside the final verdict, enabling developers to understand exactly which condition produced the outcome. This is a critical design choice for auditability — every verdict is explainable by a specific rule, not a black-box score.

2.6 Deployment Architecture

I containerized the platform using Docker and docker-compose. The Dockerfile specifies Python 3.11, installs all dependencies from requirements.txt, and exposes port 8000. Model weights are cached in a Docker volume (hf-cache) to avoid re-downloading across container restarts — important given the ~6GB size of Llama 3.2-3B weights.

I created a separate requirements_dgx.txt with GPU-specific package versions for the NVIDIA DGX cluster deployment target. The inference backend automatically selects MPS acceleration on Apple M1 (development) or CUDA on DGX (production) based on available hardware.

For Dennis's frontend integration, I configured ngrok to expose the local FastAPI server over a public HTTPS tunnel, allowing the Replit frontend to communicate with the backend during development and demo.

3. Model Backend — Llama and Claude API

3.1 Why Two Backends

The two-backend design was driven by a genuine institutional constraint: UNICC operates under data sovereignty requirements that make routing sensitive UN agency application metadata through external commercial APIs problematic for production deployments. Llama 3.2-3B-Instruct running locally on DGX addresses this constraint — all inference happens within UNICC's infrastructure.

Claude API (claude-sonnet-4-5) was added as a configurable alternative for two reasons: first, as a development and validation tool during prompt engineering; second, as a cloud deployment option for UNICC constituent organizations that do not have local GPU infrastructure. The same interface, same prompts, and same arbitration logic apply regardless of which backend is active.

3.2 LoRA Adapter — What Happened and Why

The original project plan included fine-tuning Llama-3-8B with LoRA (Low-Rank Adaptation) adapters trained on domain-specific expert datasets hosted in Council_of_Experts/Training_Data/. LoRA was planned as a way to improve the 3B model's structured JSON output quality and domain knowledge without full retraining.

Two constraints prevented completion: first, LoRA training experiments on Google Colab produced results that were not meaningfully better than base model + prompt engineering, and the training runs were unstable. Second, DGX sandbox package size constraints prevented installing the full training stack in the target deployment environment.

The decision was made to ship without adapters and achieve evaluation quality through prompt engineering and the deliberation architecture instead. The LoRA upgrade path (Llama-3-8B + adapters) remains documented as a future enhancement in the repository.

4. Key Technical Challenges and Resolutions

Challenge	Resolution
python-dotenv ImportError on clean-machine setup	Wrapped dotenv import in try/except block. Server starts successfully with or without the package installed. Fallback to system environment variables.
System uvicorn vs. venv uvicorn path conflict	Updated README run command to python -m uvicorn. Ensures inference runs within the active virtual environment where all dependencies are installed.
Claude API 401 Unauthorized on clean .env copy	Identified that cp .env.example .env copies placeholder API key. README updated with explicit instruction to set ANTHROPIC_API_KEY before running anthropic backend.
Expert agent JSON schema validation failures	Implemented failure_detected=True flag + safe default fallback for all expert output fields. Pipeline returns complete RunResponse even under partial agent failure.
LoRA fine-tuning blocked by DGX sandbox constraints	Shipped base Llama 3.2-3B-Instruct without adapters. Prompt engineering + deliberation architecture used to achieve evaluation quality. LoRA documented as future work.
Demo live inference too slow (~75s)	Pre-prepared VeriMedia output used per Professor Fortino's guidance. Live submission shown in UI, results presented from prepared output.
Sequential vs. parallel agent execution	Architecture is intentionally sequential (Governance → Threat → Behavioral) to allow downstream agents to be aware of upstream findings — improving deliberation coherence.

5. Team Integration Context

This section briefly summarizes my teammates' contributions to provide context for how my platform integrates with the full system. Their individual reports cover these areas in detail.

5.1 Wesley Koe — Council of Experts Framework

Wesley designed the conceptual framework that my platform implements: the Council of Experts architecture, the expert role definitions (Governance, Threat, Behavioral), the evaluation framework specifications referenced in each agent's system prompt, the frozen `expert_output_schema v1.0`, and the Arbitration Rules specification (Rules 1–6) that I implemented as the deterministic arbitration engine in `api.py`. The schemas and framework documents are maintained in the `Council_of_Experts/` directory of the repository.

5.2 Dennis Wu — Frontend UI

Dennis built the frontend interface at `unicc-ai-guard.replit.app`, which provides the user-facing input form (AI system name, scenario description, optional PDF upload) and the output display (expert findings, deliberation summary, final verdict, downloadable PDF report). The frontend communicates with my FastAPI backend via POST requests to the `/evaluate` endpoint, routed through the ngrok tunnel I configured during development and demo.

5.3 System Integration

The three components — my backend, Wesley's framework specifications, and Dennis's frontend — were integrated through a shared schema contract. Wesley's `expert_input_schema` and `expert_output_schema v1.0` defined the data contract between all three components. My backend enforces these schemas via Pydantic validation, ensuring that any input from Dennis's frontend and any output to it conforms to the agreed structure.

6. Individual Reflection

6.1 What I Learned

This project gave me hands-on experience with the full stack of a production AI evaluation system — from HTTP endpoint design to LLM prompt engineering to deterministic decision logic. The most valuable learning was understanding how much of an LLM system's quality comes from architecture and prompt design rather than model capability. Running a 3B parameter model with well-engineered prompts and a deliberation layer produced more consistent, explainable outputs than a naive single-call approach with a much larger model would.

I also learned the practical difference between a working demo and a deployable system. The `dotenv ImportError`, the `uvicorn` path conflict, and the 401 API key issue were all invisible during local development but would have caused immediate failures in a clean evaluation environment. Building for reproducibility — treating my own machine as 'privileged' and constantly asking 'what happens on a machine that's never seen this repo before?' — is a discipline I will carry forward.

6.2 What I Would Do Differently

If I were starting over, I would complete the LoRA fine-tuning before building the prompt engineering layer, not after. The training data Wesley prepared was well-structured, and even a partially trained adapter on the 8B model would likely have produced more reliable structured JSON outputs than prompt engineering alone. The DGX constraint was a known risk from early in the project, and I would address it earlier rather than treating it as a fallback.

I would also instrument the deliberation layer from the start with structured logging keyed by `request_id`, enabling post-hoc analysis of how often agents change positions during

critique/defense rounds and which contradiction types are most common. That data would directly inform Arbitration Rules v2.0.

6.3 Outcome

The platform was presented live at the NYU UNICC Capstone Demo Day on April 14, 2026. The VeriMedia evaluation demo ran end-to-end, the deliberation output was presented clearly, and the Reject verdict with Rule 2 trigger was explained to the panel. All core deliverables were completed within the project timeline. The platform is deployment-ready and documented for UNICC handoff.

Building a system that evaluates other AI systems for safety — and doing it with a small local model that respects data sovereignty — is exactly the kind of infrastructure the UN system needs right now. I am proud of what this platform does and confident it can be scaled.

References

- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv preprint arXiv:1606.08813.
- Bommasani, R., et al. (2021). On the opportunities and risks of foundation models. Stanford University HAI. arXiv preprint arXiv:2108.07258.
- Brundage, M., et al. (2018). The malicious use of artificial intelligence. Future of Humanity Institute. arXiv preprint arXiv:1802.07228.
- European Union Agency for Cybersecurity (ENISA). (2020). Artificial intelligence cybersecurity challenges: Threat landscape for artificial intelligence.
- Gabriel, I. (2020). Artificial intelligence, values, and alignment. *Minds and Machines*, 30(3), 411–437.
- Jobin, A., Ienca, M., & Vayena, E. (2019). The global landscape of AI ethics guidelines. *Nature Machine Intelligence*, 1(9), 389–399.
- Liang, P., et al. (2022). Holistic evaluation of language models. *Transactions on Machine Learning Research*.
- National Institute of Standards and Technology (NIST). (2023). Artificial intelligence risk management framework (AI RMF 1.0). U.S. Department of Commerce.
- Park, J. S., et al. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on UIST*.
- UNESCO. (2021). Recommendation on the ethics of artificial intelligence. United Nations Educational, Scientific and Cultural Organization.

Wachter, S., Mittelstadt, B., & Russell, C. (2017). Counterfactual explanations without opening the black box. *Harvard Journal of Law & Technology*, 31(2), 841–887.